

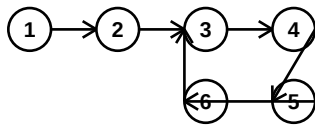
Detect a Loop in a Linked List

Personal DSA Study Notes

1. Introduction

The problem is to determine whether a singly linked list contains a loop. A loop exists when following next references eventually brings us back to a node that has already been reached.

The visual intuition is important: one next reference can point backward into an earlier part of the chain.



6 points back to 3: the next references form a cycle.

Figure 1 — A linked list whose last part points back into the list

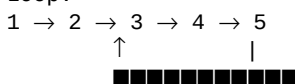
2. Problem Statement

Given the head of a linked list, return true if there is a loop and false if the list is linear and eventually reaches NULL.

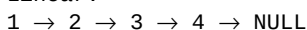
3. What Counts as a Loop?

A loop exists when there is at least one node from which traversal can eventually reach that same node again.

Loop:

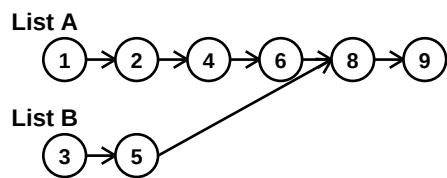


Linear:



4. Approach 1 — Hashing

Traverse the list while storing every complete node object in a hash-based structure. Before storing the current node, check whether that exact node is already present.



8 is already stored → repeated node detected.

Figure 2 — Store visited nodes and detect the repeated node reference

4.1 Pseudocode

```
create a set/map of nodes
temp = head

while temp != null:
    if temp exists in the set:
        return true
    store temp in the set
    temp = temp.next

return false
```

4.2 Original Core Algorithm

```
temp = head
while temp != null:
    if map contains temp:
        return true
    put temp in map
    temp = temp.next
return false
```

4.3 Complete Java

```
import java.util.HashSet;
import java.util.Set;

class Solution {
    public boolean hasCycle(ListNode head) {
        Set<ListNode> set = new HashSet<>();

        ListNode temp = head;

        while (temp != null) {
            if (set.contains(temp)) {
                return true;
            }

            set.add(temp);
            temp = temp.next;
        }

        return false;
    }
}
```

4.4 Complexity

Time: $O(N)$ average with hashing. Space: $O(N)$, because visited nodes are stored.

5. Why Optimize?

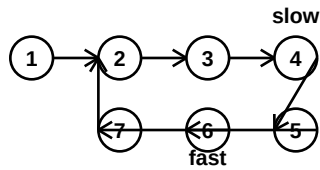
The hashing method needs external storage. The optimized method detects the loop during traversal itself, using only pointer variables.

The transcript introduces the tortoise-and-hare idea: slow moves one step and fast moves two steps.

6. Approach 2 — Slow and Fast Pointers

6.1 Pointer Movement

Both pointers start at head. In every iteration, slow advances by one node and fast advances by two nodes.

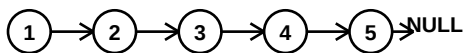


slow = 1 step, fast = 2 steps; inside the loop, fast catches slow.

Figure 3 — Slow and fast pointers moving around a loop

6.2 Linear List Case

If there is no loop, there is a last node. Fast eventually reaches NULL, so the algorithm stops.



fast reaches NULL → no loop.

Figure 4 — In a linear list, fast reaches NULL

6.3 Pseudocode

```
slow = head
fast = head

while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next

    if slow == fast:
        return true

return false
```

7. Complete Java — Optimal Approach

```
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;

            if (slow == fast) {
                return true;
            }
        }

        return false;
    }
}
```

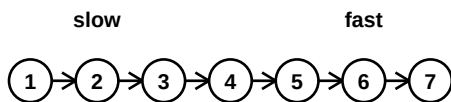
Important Java condition

```
while (fast != null && fast.next != null)
```

Check fast first. Java evaluates && from left to right, so this prevents accessing fast.next when fast is null.

8. Why Do They Meet?

Assume slow and fast are inside the loop. Let their current distance around the loop be D .



gap = D ; after one iteration the gap becomes $D - 1$.

Figure 5 — The distance between fast and slow decreases by one

Fast moves two positions while slow moves one. Therefore the gap decreases by exactly one on every iteration: $D \rightarrow D-1 \rightarrow D-2 \rightarrow \dots \rightarrow 0$.

When the distance becomes zero, $\text{slow} == \text{fast}$. That meeting proves that a loop exists.

9. Edge Cases

- Empty list $\rightarrow$ false.
- One node pointing to NULL $\rightarrow$ false.
- One node pointing to itself $\rightarrow$ true.
- Two or more nodes with no loop $\rightarrow$ false.
- Loop begins at the head $\rightarrow$ true.
- Loop begins in the middle $\rightarrow$ true.

10. Complexity Comparison

Approach	Time	Auxiliary Space
Hashing	$O(N)$ average	$O(N)$
Slow & Fast	$O(N)$	$O(1)$

11. Quick Revision

```
slow = head
fast = head

while fast != null && fast.next != null:
    slow = slow.next
    fast = fast.next.next

    if slow == fast:
        return true

return false

Time:  $O(N)$ 
Space:  $O(1)$ 
```

12. Conclusion

The important visual idea is that fast moves two nodes for every one node moved by slow. Once both are inside a loop, the distance between them decreases by one each iteration, so they must eventually meet.